

1 OF 10 >>>

STRINGBUILDER



VS

STRINGBUFFER

Every Java Developer Should Know This!



What are they?
Why do we need them?



Key Differences
Explained Simply!



Thread Safety
Which one is thread-safe?



Performance
Which one is faster?



Let's understand the **core difference** between **StringBuilder** and **StringBuffer** in the easiest way!



Swipe Left to start this comparison

2 OF 10 >>>



WHAT PROBLEM DO THEY SOLVE?

When we concatenate strings using + operator, a lot of **new objects** are created in memory.

```
1 String s = "Hello";  
2 s = s + " " + "World";  
3 s = s + "!";
```



Creates multiple **new** String objects every time.

This is very inefficient for multiple modifications.

StringBuilder and **StringBuffer** are here to solve this problem!

3 OF 10 >>>



WHY STRING IS IMMUTABLE?

String objects are immutable means once created, **cannot be changed**.

Example:

```
1 String s = "Hello";  
2 s.concat(" World");  
3 System.out.println(s); // Hello
```



Original string remains unchanged!

To modify strings efficiently, Java provides **StringBuilder** and **StringBuffer**.

4 OF 10 >>>

INTRODUCTION TO STRINGBUILDER



StringBuilder is a **mutable** class used to modify strings.

Key Points

- ✓ It is **mutable** (can be changed).
- ✓ It is **not synchronized** (not thread-safe).
- ✓ **Faster** than StringBuffer.
- ✓ Defined in **java.lang** package.



Example

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");    // Hello World  
sb.append("!");         // Hello World!
```

Output:
Hello World!



javaprograming language

5 OF 10 >>>

INTRODUCTION TO STRINGBUFFER



StringBuffer is a **mutable** class used to modify strings and it is **thread-safe**.

Key Points

- ✓ It is **mutable** (can be changed).
- ✓ It is **synchronized** (thread-safe).
- ✓ **Slower** than StringBuilder due to synchronization.
- ✓ Defined in **java.lang** package.



Example

```
StringBuffer sb = new StringBuffer("Hello");  
sb.append(" World");    // Hello World  
sb.append("!");         // Hello World!
```

Output:
Hello World!



StringBuffer is best when you need **thread-safety**.

6 OF 10 >>>

HOW STRING CONCATENATION WORKS?



When we use **+** **operator**, Java internally **converts** and **appends** strings in sequence.

Example:

```
String a = "Java";  
String b = " Programming";  
String c = a + b + " Language";  
System.out.println(c); // Output:  
// Java Programming Language
```

What happens here?

- 1 a and b are Strings.
- 2 a + b → "Java Programming"
- 3 Then + " Language" is added.
- 4 Final result is **one String**.

Rules to Remember



The **+** operator **joins** strings.



Evaluation is done from **left to right**.



If any operand is **String**, result will be **String**.



New **String** object is created in **memory**.



Concatenation is easy to use but can be **inefficient** for many operations.

7 OF 10 >>>

STRING METHODS YOU SHOULD KNOW



Java provides many **useful methods** to work with String objects.

- ✓ **length()** – Returns length of the string
- ✓ **charAt(index)** – Returns char at index
- ✓ **substring(begin, end)** – Returns substring
- ✓ **toUpperCase()** – Converts to uppercase
- ✓ **toLowerCase()** – Converts to lowercase
- ✓ **equals(str)** – Compares content
- ✓ **contains(str)** – Checks if substring exists
- ✓ **replace(old, new)** – Replaces old with new
- ✓ **trim()** – Removes leading/trailing spaces

Example:

```
String s = " Hello Java ";  
System.out.println(s.length());  
// 13  
System.out.println(s.trim());  
// Hello Java  
System.out.println(s.toUpperCase());  
// HELLO JAVA  
System.out.println(s.substring(2, 7));  
// Hello  
System.out.println(s.contains("Java"));  
// true
```



These methods make string manipulation simple and powerful. Use them smartly to write **clean** and **efficient** code.

8 OF 10 >>>

STRING COMPARISON IN JAVA



In Java, `==` compares references, while `.equals()` compares content.

COMPARISON METHODS

<code>==</code>	Checks reference equality (same memory location).
<code>.equals()</code>	Checks content equality (same characters).
<code>equalsIgnoreCase()</code>	Checks content equality, ignoring case .
<code>compareTo()</code>	Lexicographically compares two strings.

TIPS



- ✓ Always use `.equals()` to compare string content.
- ✓ `==` should be used only for reference check.

EXAMPLE

```
String a = new String("Hello");
String b = new String("Hello");

System.out.println(a == b);           // false (different references)
System.out.println(a.equals(b));       // true (same content)
System.out.println(a.equalsIgnoreCase("hello")); // true
System.out.println(a.compareTo(b));    // 0 (strings are equal)
```

Output:

```
false
true
true
0
```



Never use `==` to compare string content. It may give **unexpected results**.



javaprograming language

9 OF 10 >>>

COMMON PITFALLS & BEST PRACTICES



Using strings incorrectly can lead to **bugs** and **poor performance**.

COMMON PITFALLS

- ✗ Using `==` to compare string **content**.
- ✗ Using `+` in **loops** for string concatenation.
- ✗ Ignoring **immutability** of String objects.
- ✗ Not handling **null** strings (`NullPointerException`).

BEST PRACTICES

- ✓ Use `.equals()` or `.equalsIgnoreCase()`.
- ✓ Use `StringBuilder/StringBuffer` in loops.
- ✓ Remember: Strings are **immutable**.
- ✓ Always check for **null** before use.

EXAMPLE: AVOID + IN LOOPS

✗ Inefficient (Slow)

```
String result = "";
for (int i = 1; i <= 10000; i++) {
    result = result + i; // Creates
                        // many objects
}
```

✓ Efficient (Fast)

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 10000; i++) {
    sb.append(i); // Reuses the same object
}
String result = sb.toString();
```

Why?

- `+` creates new `String` objects every time.
- `StringBuilder` reuses the same object.



Write **clean code** by avoiding common mistakes and following **best practices**!



javaprograming language

10 OF 10 >>>

SUMMARY: KEY TAKEAWAYS



Strings are **powerful** in Java when used correctly. Choose the right methods and follow best practices to write **clean**, **efficient**, and **bug-free** code.

WHAT WE LEARNED

- ✓ String and **StringBuffer** basics
- ✓ How String **concatenation** works
- ✓ Important **String** methods
- ✓ String **comparison** in Java
- ✓ Common **pitfalls** and how to avoid them
- ✓ **Best practices** for using strings

WHEN TO USE WHAT?

Use	When?	Example
String	For fixed/unmodifiable text (immutable)	<code>String s = "Hello";</code>
StringBuffer	For mutable text with thread-safety	<code>StringBuffer sb = new StringBuffer();</code>
StringBuilder	For mutable text (single-threaded, fast performance)	<code>StringBuilder sb = new StringBuilder();</code>



Keep Practicing!

The more you code, the better you understand.



That's a wrap! 🎉 You're now ready to use **Strings** like a pro!



javaprograming language